

Q-1) Perform the following operations using MongoDB:

1. Create: Insert multiple documents into a collection called employees. Each document should have the following fields: name, position, and salary.

Insert the following employees:

- a. name: "Alice", position: "Manager", salary: 70000
- b. name: "Bob", position: "Developer", salary: 55000
- c. name: "Charlie", position: "HR", salary: 45000

```
test> db.employees.insertMany([
...   { name: "Alice", position: "Manager", salary: 70000 },
...   { name: "Bob", position: "Developer", salary: 55000 },
...   { name: "Charlie", position: "HR", salary: 45000 }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68b7cfb94909c9bcc1735189'),
    '1': ObjectId('68b7cfb94909c9bcc173518a'),
    '2': ObjectId('68b7cfb94909c9bcc173518b')
  }
}
```

2. Read: Query the employees collection to find all employees with a salary greater than 50000.

```
test> db.employees.find({ salary: { $gt: 50000 } })
[
  {
    _id: ObjectId('68b7cfb94909c9bcc1735189'),
    name: 'Alice',
    position: 'Manager',
    salary: 70000
  },
  {
    _id: ObjectId('68b7cfb94909c9bcc173518a'),
    name: 'Bob',
    position: 'Developer',
    salary: 55000
  }
]
```

3. Update: Increase the salary of all employees whose position is "Developer" by 10%.

```
test> db.employees.updateMany(
...   { position: "Developer" },
...   { $mul: { salary: 1.1 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
test> db.employees.find().pretty()
[
  {
    _id: ObjectId('68b7cfb94909c9bcc1735189'),
    name: 'Alice',
    position: 'Manager',
    salary: 70000
  },
  {
    _id: ObjectId('68b7cfb94909c9bcc173518a'),
    name: 'Bob',
    position: 'Developer',
    salary: 66550.00000000001
  },
  {
    _id: ObjectId('68b7cfb94909c9bcc173518b'),
    name: 'Charlie',
    position: 'HR',
    salary: 45000
  }
]
```

4. Delete: Remove the employee named "Charlie" from the employees collection.

```
test> db.employees.deleteOne({ name: "Charlie" })
{ acknowledged: true, deletedCount: 1 }
test> db.employees.find().pretty()
[
  {
    _id: ObjectId('68b7d3680b46480e4b735189'),
    name: 'Alice',
    position: 'Manager',
    salary: 70000
  },
  {
    _id: ObjectId('68b7d3680b46480e4b73518a'),
    name: 'Bob',
    position: 'Developer',
    salary: 55000
  }
]
```

Q-2 Write a MongoDB script to perform the following CRUD operations:

1) Create: Insert a new document into a collection called students. The document should include the following fields: name, age, and course.

Insert a document with values:

name: "John Doe"

age: 22

course: "Computer Science"

```
studb> db.students.find().pretty()
[
  {
    _id: ObjectId('68b7d514bc59b1f77a73518a'),
    name: 'John Doe',
    age: 22,
    course: 'Computer Science'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518b'),
    name: 'Jane Smith',
    age: 25,
    course: 'Data Science'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518c'),
    name: 'Raj Patel',
    age: 20,
    course: 'Mechanical Engineering'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518d'),
    name: 'Sara Khan',
    age: 18,
    course: 'Electrical Engineering'
  }
]
```

2) Read: Retrieve all documents from the students collection where the age is greater than 20.

```
studb> db.students.find({
...   age: { $gt: 20 }
... })
[
  {
    _id: ObjectId('68b7d514bc59b1f77a73518a'),
    name: 'John Doe',
    age: 22,
    course: 'Computer Science'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518b'),
    name: 'Jane Smith',
    age: 25,
    course: 'Data Science'
  }
]
```

3) Update: Update the course field of the student named "John Doe" to "Data Science".

```
studb> db.students.updateOne(
...   { name: "John Doe" },
...   { $set: { course: "Data Science" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studb> db.students.find().pretty()
[
  {
    _id: ObjectId('68b7d514bc59b1f77a73518a'),
    name: 'John Doe',
    age: 22,
    course: 'Data Science'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518b'),
    name: 'Jane Smith',
    age: 25,
    course: 'Data Science'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518c'),
    name: 'Raj Patel',
    age: 20,
    course: 'Mechanical Engineering'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518d'),
    name: 'Sara Khan',
    age: 18,
    course: 'Electrical Engineering'
  }
]
```

4) Delete: Delete all documents from the students collection where the age is less than 21.

```
studb> db.students.deleteMany({
...   age: { $lt: 21 }
... })
{ acknowledged: true, deletedCount: 2 }
studb> db.students.find().pretty()
[
  {
    _id: ObjectId('68b7d514bc59b1f77a73518a'),
    name: 'John Doe',
    age: 22,
    course: 'Data Science'
  },
  {
    _id: ObjectId('68b7d514bc59b1f77a73518b'),
    name: 'Jane Smith',
    age: 25,
    course: 'Data Science'
  }
]
```

Q-3 Write MongoDB commands to perform the following operations on a collection named books:

1) Create: Insert a document representing a book with the fields title, author, publishedYear, and copiesSold. Use the following data:

title: "The Alchemist"
author: "Paulo Coelho"
publishedYear: 1988
copiesSold: 65000000

```
bookcoll> db.books.find().pretty()
[
  {
    _id: ObjectId('68b7db7edf58026761735189'),
    title: 'To Kill a Mockingbird',
    author: 'Harper Lee',
    publishedYear: 1960,
    copiesSold: 40000
  },
  {
    _id: ObjectId('68b7db7edf5802676173518a'),
    title: '1984',
    author: 'George Orwell',
    publishedYear: 1949,
    copiesSold: 320000
  },
  {
    _id: ObjectId('68b7db7edf5802676173518b'),
    title: 'The Da Vinci Code',
    author: 'Dan Brown',
    publishedYear: 2003,
    copiesSold: 80000000
  },
  {
    _id: ObjectId('68b7db7edf5802676173518c'),
    title: 'The Great Gatsby',
    author: 'F. Scott Fitzgerald',
    publishedYear: 1925,
    copiesSold: 25000000
  },
  {
    _id: ObjectId('68b7db7edf5802676173518d'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    publishedYear: 1988,
    copiesSold: 65000000
  }
]
```

2) Read: Find all books in the books collection that were published before the year 2000.

```
bookcoll> db.books.find({
...   publishedYear: { $lt: 2000 }
... })
[
  {
    _id: ObjectId('68b7db7edf58026761735189'),
    title: 'To Kill a Mockingbird',
    author: 'Harper Lee',
    publishedYear: 1960,
    copiesSold: 40000
  },
  {
    _id: ObjectId('68b7db7edf5802676173518a'),
    title: '1984',
    author: 'George Orwell',
    publishedYear: 1949,
    copiesSold: 32000
  },
  {
    _id: ObjectId('68b7db7edf5802676173518c'),
    title: 'The Great Gatsby',
    author: 'F. Scott Fitzgerald',
    publishedYear: 1925,
    copiesSold: 2500000
  },
  {
    _id: ObjectId('68b7db7edf5802676173518d'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    publishedYear: 1988,
    copiesSold: 6500000
  }
]
```

3. Update: Add a new field rating with a value of 4.8 to the document where the title is "The Alchemist".

```
bookcoll> db.books.updateOne(
...   { title: "The Alchemist" },
...   { $set: { rating: 4.8 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

```
}  
{  
  _id: ObjectId('68b7db7edf5802676173518d'),  
  title: 'The Alchemist',  
  author: 'Paulo Coelho',  
  publishedYear: 1988,  
  copiesSold: 65000000,  
  rating: 4.8  
}
```

4. Delete all books from the collection where copiesSold is less than

1 million.

```
bookcoll> db.books.deleteMany({ copiesSold: { $lt: 1000000 } })  
{ acknowledged: true, deletedCount: 0 }  
bookcoll>
```

4) Program to calculate average weather temperature of city from csv file.**File : ATCalc**

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ATCalc
{
    public static void main(String[] args) throws Exception
    {
        Configuration conf=new Configuration();
        Job job=Job.getInstance(conf,"Average Temperature");

        job.setJarByClass(ATCalc.class);
        job.setMapperClass(ATMapper.class);
        job.setReducerClass(ATReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job,new Path(args[0]));
        FileOutputFormat.setOutputPath(job,new Path(args[1]));

        System.exit(job.waitForCompletion(true)?0:1);
    }
}
```

File:ATMapper

```
import java.io.IOException;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.Mapper;

public class ATMapper extends Mapper<Object, Text, Text, IntWritable>
{
    private Text city=new Text();
```



```
private IntWritable temperature=new IntWritable();
```

```
@Override
```

```
public void map (Object key, Text value, Context context) throws IOException, InterruptedException
```

```
{
    String[] parts=value.toString().split(",");
    if (parts.length==3)
    {
        city.set(parts[0]);
        temperature.set(Integer.parseInt(parts [2]));
        context.write(city, temperature);
    }
}
}
```

File:ATReducer

```
import java.io.IOException;
```

```
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.io.IntWritable;
```

```
import org.apache.hadoop.mapreduce.Reducer;
```

```
public class ATReducer extends Reducer<Text,IntWritable,Text,IntWritable>
```

```
{
    private IntWritable result=new IntWritable();

    @Override
    public void reduce(Text key,Iterable<IntWritable> values,Context context) throws
    IOException,InterruptedException
    {
        int sum=0;
        int count=0;
        for(IntWritable val:values)
        {
            sum+=val.get();
            count++;
        }
        int average=sum/count;
        result.set(average);
        context.write(key,result);
    }
}
```

}

Output:

```
C:\Windows\System32\cmd.exe
D:\BDHexample>cd output27

D:\BDHexample\output27>dir
Volume in drive D has no label.
Volume Serial Number is BCA5-1040

Directory of D:\BDHexample\output27

01-08-2025  11:24    <DIR>          .
01-08-2025  11:24    <DIR>          ..
01-08-2025  11:23             12 .part-r-00000.crc
01-08-2025  11:24              8 . _SUCCESS.crc
01-08-2025  11:23             19 part-r-00000
01-08-2025  11:24              0 _SUCCESS
               4 File(s)              39 bytes
               2 Dir(s)  210,748,833,792 bytes free

D:\BDHexample\output27>type part-r-00000
jaipur  30
surat   42

D:\BDHexample\output27>
D:\BDHexample\output27>
```

5) Write a MapReduce program that reads a CSV file containing sales transactions with columns: transaction_id, product_id, quantity, and price. Calculate the total revenue generated for each product and output it as product_id, total_revenue.

SalesRevenueCalc

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.FloatWritable; // Use FloatWritable for revenue
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class SalesRevenueCalc {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Product Revenue Calculation");

        job.setJarByClass(SalesRevenueCalc.class);
        job.setMapperClass(SalesMapper.class);
        job.setReducerClass(SalesReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(FloatWritable.class); // Output revenue as FloatWritable

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

SalesMapper

```
import java.io.IOException;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.FloatWritable; // Using FloatWritable for revenue
import org.apache.hadoop.mapreduce.Mapper;

public class SalesMapper extends Mapper<Object, Text, Text, FloatWritable> {
```

```
private Text product_id = new Text();
private FloatWritable revenue = new FloatWritable();

@Override
public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
    String[] parts = value.toString().split(","); // Assuming comma-separated values

    // Ensure the line has the expected number of columns (transaction_id, product_id, quantity, price)
    if (parts.length == 4) {
        try {
            String pId = parts[1]; // product_id is the second column (index 1)
            int quantity = Integer.parseInt(parts[2]); // quantity is the third column (index 2)
            float price = Float.parseFloat(parts[3]); // price is the fourth column (index 3)

            product_id.set(pId);
            revenue.set(quantity * price); // Calculate revenue for the transaction

            context.write(product_id, revenue); // Emit product_id and revenue
        } catch (NumberFormatException e) {
            // Handle cases where quantity or price are not valid numbers
            System.err.println("Skipping bad record: " + value.toString() + " - " + e.getMessage());
        }
    }
}
```

SalesReducer

```
import java.io.IOException;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.mapreduce.Reducer;

public class SalesReducer extends Reducer<Text, FloatWritable, Text, FloatWritable> {
    private FloatWritable totalRevenue = new FloatWritable();

    @Override
    public void reduce(Text key, Iterable<FloatWritable> values, Context context)
        throws IOException, InterruptedException {
        float sumOfRevenue = 0.0f;
```

```
// Iterate through all the revenue values for the current product_id
for (FloatWritable value : values) {
    sumOfRevenue += value.get(); // Sum up the revenues
}

totalRevenue.set(sumOfRevenue); // Set the total revenue
context.write(key, totalRevenue); // Emit product_id and total revenue
}
}
```

Output:

```
D:\MapReduceProj>cd output
D:\MapReduceProj\output>dir
Volume in drive D has no label.
Volume Serial Number is DA2E-71B0

Directory of D:\MapReduceProj\output

01-08-2025  12:39    <DIR>          .
01-08-2025  12:39    <DIR>          ..
01-08-2025  12:39                12 .part-r-00000.crc
01-08-2025  12:39                8  _SUCCESS.crc
01-08-2025  12:39               94 part-r-00000
01-08-2025  12:39                0 _SUCCESS
               4 File(s)              114 bytes
               2 Dir(s)  218,693,652,480 bytes free

D:\MapReduceProj\output>type part-r-00000
1      8750.0
10     16500.0
2      1500.0
3       9000.0
4      9540.0
5       6006.0
6      2882.0
7       6000.0
8      16000.0
9      14400.0

D:\MapReduceProj\output>
```

6) Given a CSV file with columns: movie_id, user_id, rating, write a MapReduce program to calculate the average rating for each movie.

Output the results as movie_id, average_rating.

movieCalc

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class movieCalc {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: Calc <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Movie Rating Average");

        job.setJarByClass(movieCalc.class);
        job.setMapperClass(movieMapper.class);
        job.setReducerClass(movieReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(DoubleWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

movieMapper

```
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class movieMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {

    private Text movieId = new Text();

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String line = value.toString();
        String[] fields = line.split(",");

        if (fields.length == 3) {
            String movie_id = fields[0].trim();
            String ratingStr = fields[2].trim();
            try {
                double rating = Double.parseDouble(ratingStr);
                movieId.set(movie_id);
                context.write(movieId, new DoubleWritable(rating));
            } catch (NumberFormatException e) {
                // skip invalid ratings
            }
        }
    }
}
```

movieReducer

```
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class movieReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
```

```
@Override
protected void reduce(Text key, Iterable<DoubleWritable> values, Context context)
    throws IOException, InterruptedException {
    double sum = 0;
    int count = 0;

    for (DoubleWritable val : values) {
        sum += val.get();
        count++;
    }
    if (count > 0) {
        double average = sum / count;
        context.write(key, new DoubleWritable(average));
    }
}
```

Output:-

```
D:\20\BDHExample\journal>cd output11
D:\20\BDHExample\journal\output11>dir
Volume in drive D has no label.
Volume Serial Number is 62C5-E7DA

Directory of D:\20\BDHExample\journal\output11

04-08-2025  11:29    <DIR>          .
04-08-2025  11:29    <DIR>          ..
04-08-2025  11:29                12 .part-r-00000.crc
04-08-2025  11:29                8 ._SUCCESS.crc
04-08-2025  11:29               33 part-r-00000
04-08-2025  11:29                0 _SUCCESS
                4 File(s)              53 bytes
                2 Dir(s)  207,008,681,984 bytes free

D:\20\BDHExample\journal\output11>type part-r-00000
jungle book      4.0
pk               4.0
saaho            4.0
```


7) Calculating Maximum and Minimum Temperature

Using a weather dataset with columns: city, date, temperature, write a MapReduce program to find the maximum and minimum temperature recorded for each city.
Output the results as city, max_temperature, min_temperature.

MaxMinTempCalc

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxMinTempCalc {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Max Min Temperature");

        job.setJarByClass(MaxMinTempCalc.class);
        job.setMapperClass(MaxMinTempMapper.class);
        job.setReducerClass(MaxMinTempReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(FloatWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

MaxMinTempMapper

```
import java.io.IOException;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
```

```
public class MaxMinTempMapper extends Mapper<LongWritable, Text, Text, FloatWritable> {

    private Text city = new Text();
    private FloatWritable temperature = new FloatWritable();

    public void map(LongWritable key, Text value, Context context) throws IOException,
    InterruptedException {
        String line = value.toString().trim();
        if (line.startsWith("city")) return; // Skip header

        String[] parts = line.split(",");
        if (parts.length == 3) {
            try {
                city.set(parts[0].trim());
                temperature.set(Float.parseFloat(parts[2].trim()));
                context.write(city, temperature);
            } catch (NumberFormatException e) {
                System.err.println("Invalid temperature: " + line);
            }
        } else {
            System.err.println("Invalid line format: " + line);
        }
    }
}
```

MaxMinTempReducer

```
import java.io.IOException;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class MaxMinTempReducer extends Reducer<Text, FloatWritable, Text, Text> {

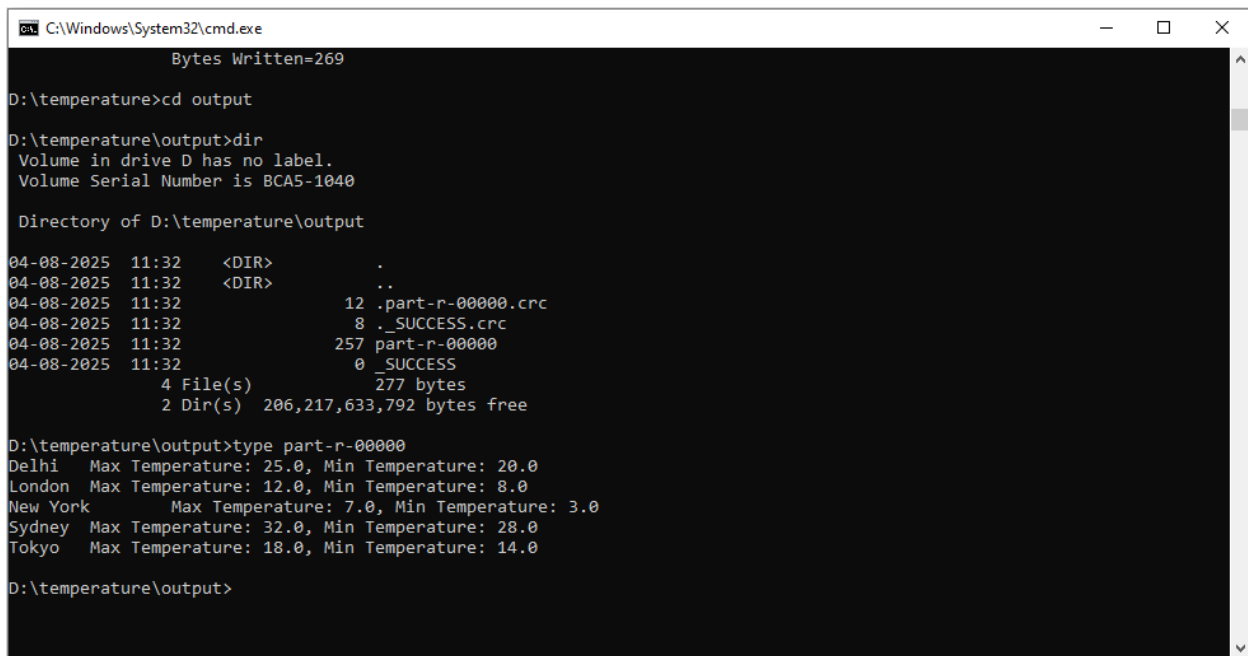
    private Text result = new Text();

    public void reduce(Text key, Iterable<FloatWritable> values, Context context) throws
    IOException, InterruptedException {
        float maxTemp = Float.MIN_VALUE;
        float minTemp = Float.MAX_VALUE;

        for (FloatWritable val : values) {
```

```
float temp = val.get();  
maxTemp = Math.max(maxTemp, temp);  
minTemp = Math.min(minTemp, temp);  
}  
  
result.set("Max Temperature: " + maxTemp + ", Min Temperature: " + minTemp);  
context.write(key, result);  
}  
}
```

Output:-



```
C:\Windows\System32\cmd.exe  
Bytes Written=269  
D:\temperature>cd output  
D:\temperature\output>dir  
Volume in drive D has no label.  
Volume Serial Number is BCA5-1040  
  
Directory of D:\temperature\output  
  
04-08-2025 11:32 <DIR> .  
04-08-2025 11:32 <DIR> ..  
04-08-2025 11:32 12 .part-r-00000.crc  
04-08-2025 11:32 8 ._SUCCESS.crc  
04-08-2025 11:32 257 part-r-00000  
04-08-2025 11:32 0 _SUCCESS  
4 File(s) 277 bytes  
2 Dir(s) 206,217,633,792 bytes free  
  
D:\temperature\output>type part-r-00000  
Delhi Max Temperature: 25.0, Min Temperature: 20.0  
London Max Temperature: 12.0, Min Temperature: 8.0  
New York Max Temperature: 7.0, Min Temperature: 3.0  
Sydney Max Temperature: 32.0, Min Temperature: 28.0  
Tokyo Max Temperature: 18.0, Min Temperature: 14.0  
D:\temperature\output>
```

8. Create a database named `company_db` and within it, create a table called `employees` with the following columns: `employee_id` (INT), `name` (STRING), `department` (STRING), and `salary` (FLOAT). Ensure the table stores data in text format.

```
OK
105    Eve Davis      HR      62000.0
104    David Lee     IT      28000.0
102    Bob Johnson   IT      75000.0
101    Alice Smith   HR      60000.0
103    Charlie Brown Finance 55000.0
Time taken: 0.239 seconds, Fetched: 5 row(s)
```

Query:

1. Write a Hive query to retrieve all employee details from the `employees` table where the department is IT.

```
hive> SELECT *
> FROM employees
> WHERE department = 'IT';
```

Output:

```
OK
102    Bob Johnson   IT      75000.0
Time taken: 0.124 seconds, Fetched: 1 row(s)
```

2. Write a Hive query to delete all records from the `employees` table where the salary is below 30000.

Original Data

```
OK
105    Eve Davis      HR      62000.0
104    David Lee     IT      28000.0
102    Bob Johnson   IT      75000.0
101    Alice Smith   HR      60000.0
103    Charlie Brown Finance 55000.0
Time taken: 0.239 seconds, Fetched: 5 row(s)
```

```
hive> DELETE FROM employees
> WHERE salary < 30000;
```

Output:

```
OK
105    Eve Davis      HR      62000.0
102    Bob Johnson   IT      75000.0
101    Alice Smith   HR      60000.0
103    Charlie Brown Finance 55000.0
Time taken: 0.174 seconds, Fetched: 4 row(s)
```

3. Update the location column for all employees whose department is HR to Head Office.

```
hive> UPDATE employees
> SET location = 'Head Office'
> WHERE department = 'HR';
```

Output:

```
103    Charlie Brown   Finance 55000.0 NULL
102    Bob Johnson    IT      75000.0 London
105    Eve Davis      HR      62000.0 Head Office
101    Alice Smith    HR      60000.0 Head Office
Time taken: 0.102 seconds, Fetched: 4 row(s)
```

4. Delete the department record where department_id = 3.

```
OK
103    Charlie Brown   Finance 55000.0 NULL
102    Bob Johnson    IT      75000.0 London
105    Eve Davis      HR      62000.0 Head Office
101    Alice Smith    HR      60000.0 Head Office
Time taken: 0.094 seconds, Fetched: 4 row(s)
```

```
hive> DELETE FROM employees
> WHERE employee_id = 103;
```

Output:

```
OK
102    Bob Johnson    IT      75000.0 London
105    Eve Davis      HR      62000.0 Head Office
101    Alice Smith    HR      60000.0 Head Office
Time taken: 0.141 seconds, Fetched: 3 row(s)
```

9. Create a database named `university_db` and within it, create a table called `students` with the following columns: `student_id` (INT), `name` (STRING), `course` (STRING), and `marks` (FLOAT). Ensure the table stores data in text format.

```
OK
101    Aarav Mehta    Computer Science    78.5
102    Isha Patel    Mathematics    65.0
103    Rohan Desai    Physics    49.0
104    Sneha Shah    Computer Science    88.0
105    Kunal Joshi    Mathematics    55.5
106    Neha Verma    Biology    45.0
107    Devansh Singh    Computer Science    92.0
108    Ananya Rao    Mathematics    72.0
```

Query:

1. Write a Hive query to retrieve all student details from the `students` table where the course is Computer Science.

Output:

```
hive> SELECT * FROM students WHERE course = 'Computer Science';
2025-08-21T09:53:17,933 INFO [main] org.apache.hadoop.hive.conf.HiveConf: Property hive.metastore.local is deprecated.
3c2ec749-4f02-40e3-8aa4-b57114c709ea
2025-08-21T09:53:17,933 INFO [main] org.apache.hadoop.hive.ql.session.SessionState: Session 3c2ec749-4f02-40e3-8aa4-b57114c709ea main
2025-08-21T09:53:17,989 INFO [3c2ec749-4f02-40e3-8aa4-b57114c709ea main] org.apache.hadoop.hive.ql.session.SessionState: Creating directory if it doesn't exist: file:/C:/Users/student/AppData/Local/Temp/hive_2025-08-21_09-53-17_944_4721158586692786381-1/-mr-10001/.hive-warehouse
86381-1
OK
101    Aarav Mehta    Computer Science    78.5
104    Sneha Shah    Computer Science    88.0
107    Devansh Singh    Computer Science    92.0
Time taken: 0.081 seconds, Fetched: 3 row(s)
```

2. Write a Hive query to delete all records from the `students` table where the marks are below 50.

```
hive> DELETE FROM students WHERE marks < 50;
```

Output:

```
OK
101  Aarav Mehta    Computer Science    78.5
102  Isha Patel     Mathematics         65.0
104  Sneha Shah     Computer Science    88.0
105  Kunal Joshi     Mathematics         55.5
107  Devansh Singh   Computer Science    92.0
108  Ananya Rao      Mathematics         72.0
Time taken: 0.097 seconds, Fetched: 6 row(s)
```

3. Update the marks column for all students whose course is Mathematics by adding 10 bonus points.

```
hive> UPDATE students SET marks = marks + 10 WHERE course = 'Mathematics';
```

Output:

```
OK
101  Aarav Mehta    Computer Science    78.5
104  Sneha Shah     Computer Science    88.0
107  Devansh Singh   Computer Science    92.0
105  Kunal Joshi     Mathematics         65.5
102  Isha Patel     Mathematics         75.0
108  Ananya Rao      Mathematics         82.0
Time taken: 0.069 seconds, Fetched: 6 row(s)
```

4. Delete the record from the students table where the student_id = 102.

```
hive> DELETE FROM students WHERE student_id = 102;
```

Output:

```
OK
101  Aarav Mehta    Computer Science    78.5
104  Sneha Shah     Computer Science    88.0
107  Devansh Singh   Computer Science    92.0
105  Kunal Joshi     Mathematics         65.5
108  Ananya Rao      Mathematics         82.0
Time taken: 0.078 seconds, Fetched: 5 row(s)
```

10. Create a database named `retail_db` and within it, create a table called `products` with the following columns: `product_id` (INT), `product_name` (STRING), `category` (STRING), and `price` (FLOAT). Ensure the table stores data in text format.

```
OK
2      Laptop  Electronics      799.99
4      Table   Furniture        150.0
1      Smartphone Electronics    299.99
3      Chair   Furniture        89.99
5      Headphones Electronics    49.99
Time taken: 0.16 seconds, Fetched: 5 row(s)
```

Query:

1. Write a Hive query to retrieve all product details from the `products` table where the category is Electronics.

```
hive> SELECT * FROM products
> WHERE category = 'Electronics';
```

Output:

```
OK
2      Laptop  Electronics      799.99
1      Smartphone Electronics    299.99
5      Headphones Electronics    49.99
```

2. Write a Hive query to delete all records from the `products` table where the price is below 100.

```
hive> DELETE FROM products
> WHERE price < 100;
```

Output:

```
OK
2      Laptop  Electronics      799.99
4      Table   Furniture        150.0
1      Smartphone Electronics    299.99
Time taken: 0.145 seconds, Fetched: 3 row(s)
```

3. Update the price column for all products in the Furniture category by increasing it by 5%.

```
hive> UPDATE products
> SET price = price * 1.05
> WHERE category = 'Furniture';
```


Output:

```
OK
2      Laptop Electronics      799.99
1      Smartphone      Electronics      299.99
4      Table      Furniture      157.5
Time taken: 0.101 seconds, Fetched: 3 row(s)
```

4. Delete the product record from the products table where the product_id = 5.

```
OK
2      Laptop Electronics      799.99
4      Table      Furniture      150.0
1      Smartphone      Electronics      299.99
3      Chair      Furniture      89.99
5      Headphones      Electronics      49.99
Time taken: 0.16 seconds, Fetched: 5 row(s)
```

```
hive> DELETE FROM products
> WHERE product_id = 5;
```

Output:

```
OK
2      Laptop Electronics      799.99
1      Smartphone      Electronics      299.99
4      Table      Furniture      157.5
3      Chair      Furniture      89.99
Time taken: 0.115 seconds, Fetched: 4 row(s)
```

11. Create a database named `hospital_db` and within it, create a table called `patients` with the following columns: `patient_id` (INT), `patient_name` (STRING), `age` (INT), and `disease` (STRING). Ensure the table stores data in text format.

```
OK
7      Ravi Kumar      16      Cold
8      Meena Iyer      40      Diabetes
5      Karan Joshi     52      Hypertension
1      Amit Patel      45      Diabetes
6      Neha Verma      25      Flu
4      Priya Desai     29      Diabetes
3      Rahul Mehta     17      Asthma
2      Sneha Shah      32      Hypertension
Time taken: 0.142 seconds, Fetched: 8 row(s)
```

Query:

1. Write a Hive query to retrieve all patient details from the `patients` table where the disease is Diabetes.

```
hive> SELECT * FROM patients
> WHERE disease = 'Diabetes';
```

Output:

```
OK
8      Meena Iyer      40      Diabetes
1      Amit Patel      45      Diabetes
4      Priya Desai     29      Diabetes
Time taken: 0.131 seconds, Fetched: 3 row(s)
```

2. Write a Hive query to delete all records from the `patients` table where the age is below 18.

```
hive> DELETE FROM patients
> WHERE age < 18;
```

Output:

```
OK
8      Meena Iyer      40      Diabetes
5      Karan Joshi     52      Hypertension
1      Amit Patel      45      Diabetes
6      Neha Verma      25      Flu
4      Priya Desai     29      Diabetes
2      Sneha Shah      32      Hypertension
Time taken: 0.097 seconds, Fetched: 6 row(s)
```

3. Update the disease column for all patients whose disease is Hypertension to Chronic Hypertension.

```
hive> UPDATE patients
> SET disease = 'Chronic Hypertension'
> WHERE disease = 'Hypertension';
```

Output:

```
OK
8      Meena Iyer      40      Diabetes
1      Amit Patel     45      Diabetes
6      Neha Verma     25      Flu
4      Priya Desai    29      Diabetes
5      Karan Joshi    52      Chronic Hypertension
2      Sneha Shah     32      Chronic Hypertension
Time taken: 0.109 seconds, Fetched: 6 row(s)
```

4. Delete the record from the patients table where the patient_id = 8.

```
OK
8      Meena Iyer      40      Diabetes
1      Amit Patel     45      Diabetes
6      Neha Verma     25      Flu
4      Priya Desai    29      Diabetes
5      Karan Joshi    52      Chronic Hypertension
2      Sneha Shah     32      Chronic Hypertension
Time taken: 0.109 seconds, Fetched: 6 row(s)
```

```
hive> DELETE FROM patients
> WHERE patient_id = 8;
```

Output:

```
OK
1      Amit Patel     45      Diabetes
6      Neha Verma     25      Flu
4      Priya Desai    29      Diabetes
5      Karan Joshi    52      Chronic Hypertension
2      Sneha Shah     32      Chronic Hypertension
Time taken: 0.142 seconds, Fetched: 5 row(s)
```